Disk Latency Monitor

To create a proper Read/Write Disk Latency Monitor you need to have a collector that collects data from an interval and compares breaks down the time (in milliseconds) which SQL doesn't naturally do. So in order to get accurate latency intervals, you need to build a collector table, a threshold table, and 3 jobs that work together to get more real time results as by default SQL only stores averages since last SQL server restart which for real time monitoring, isn't very effective.

IMPORTANT NOTE: After building or clearing the tables it will take a couple job collection runs in order for data to populate and be evaluated, so don't panic if it doesn't seem like it's working at first, just give it a few runs.

Step 1 - Create Tables and Indexes

First you'll need to create 2 tables, you can create these anywhere you want, I chose to put them in msdb because they are lightweight but you could also put them in a maintenance database. The first table is a snapshot table, which collects and stores the initial I/O stats at job run time, it has a couple indexes to make the table more efficient. Second table is where alert history is stored when the Monitors thresholds are met, this keeps a track record of latency breaches and tracks patterns and ongoing events to help signal if a Latency issue is Critical or a Warning. It too has an Index to help with efficiency. Run the below script to build the tables…

```
USE msdb;
GO

SET NOCOUNT ON;
GO

/*=============================================================
  Snapshot table
=============================================================*/
IF OBJECT_ID('dbo.DiskLatencySnapshots','U') IS NULL
BEGIN
    CREATE TABLE dbo.DiskLatencySnapshots
    (
        SnapshotID bigint IDENTITY(1,1) NOT NULL
            CONSTRAINT PK_DiskLatencySnapshots PRIMARY KEY,
        SnapshotTime datetime2(0) NOT NULL,
        database_id int NOT NULL,
        file_id int NOT NULL,
        database_name sysname NOT NULL,
        logical_name sysname NOT NULL,
        physical_name nvarchar(260) NOT NULL,
        type_desc nvarchar(60) NOT NULL,
        num_of_reads bigint NOT NULL,
        io_stall_read_ms bigint NOT NULL,
        num_of_writes bigint NOT NULL,
        io_stall_write_ms bigint NOT NULL,
        size_on_disk_bytes bigint NULL,
        sample_ms bigint NULL
    );

    CREATE INDEX IX_DiskLatencySnapshots_DbFileTime
        ON dbo.DiskLatencySnapshots(database_id, file_id, SnapshotTime DESC, SnapshotID DESC);

    CREATE INDEX IX_DiskLatencySnapshots_SnapshotTime
        ON dbo.DiskLatencySnapshots(SnapshotTime);
END
GO


/*=============================================================
  Alert history table
=============================================================*/
IF OBJECT_ID('dbo.DiskLatencyAlertHistory','U') IS NULL
BEGIN
```

```
    CREATE TABLE dbo.DiskLatencyAlertHistory
    (
      AlertHistoryID              bigint IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_DiskLatencyAlertHistory PRIMARY KEY,
      AlertTime                   datetime2(0) NOT NULL
        CONSTRAINT DF_DiskLatencyAlertHistory_AlertTime DEFAULT (SYSDATETIME()),
      database_name               sysname NOT NULL,
      logical_name                sysname NOT NULL,
      type_desc                   nvarchar(60) NOT NULL,
      breach_type                 nvarchar(10) NOT NULL,   -- READ / WRITE / BOTH
      avg_read_ms                 decimal(18,2) NULL,
      avg_write_ms                decimal(18,2) NULL,
      reads_in_interval           bigint NULL,
      writes_in_interval          bigint NULL,
      WasAlerted                  bit NOT NULL
        CONSTRAINT DF_DiskLatencyAlertHistory_WasAlerted DEFAULT (0),
      Severity                    varchar(20) NULL,       -- WARNING / CRITICAL
      ConsecutiveBreachesRequired    int NULL,
      CooldownMinutes             int NULL,
      WarningWindowMinutes        int NULL,
      CriticalReadThresholdMs     int NULL,
      CriticalWriteThresholdMs    int NULL
    );

    CREATE INDEX IX_DiskLatencyAlertHistory_Search
      ON dbo.DiskLatencyAlertHistory
      (
        database_name,
        logical_name,
        type_desc,
        breach_type,
        AlertTime DESC
      )
      INCLUDE
      (
        WasAlerted,
        Severity
      );
END
GO
```

Step 2 - Create SQL Jobs

You need to make sure the tables are in place first as these jobs will not work without them, we'll be creating 3 jobs that work in tandem and setup schedules for each. The jobs are as follows…

Disk Latency Snapshot Collector - This runs every 5 minutes and collects the initial I/O data from every database on the server.
Disk Latency Alert Monitor - This runs every 5 minutes and processes the data collected from the Collector and compares it to set thresholds and sends an alert if those thresholds are breached.
Disk Latency Snapshot Cleanup - This runs daily at 12:15AM and cleans up both the tables to prevent bloat. By default it deletes anything on the Snapshot table older than 14 days, and anything older than 30 days on the Alert History table.

The first job is the Snapshot Collector, what this does is take a I/O snapshot of all databases every 5 minutes and then stores that information into the DiskLatencySnapshots table to be used for comparative purposes in between collector runs, this way you can get a more granular reading on Latency activity within a given period of time.

The second job is the bigger one, and that's the Latency Alert Monitor. What this does is it runs a check on the data collected from the Snapshot collector and compares it to user defined thresholds, and then either flags it as a warning (something to keep an eye on for possible pending Latency issues), or flags it as Critical (this needs to be looked at now)and stores all the I/O details in the DiskLatencyAlertHistory table. All of this can be controlled by modifying several threshold values that are highlighted below in the script, but we'll go into each threshold briefly now.

> **@DefaultReadLatencyMs and @DefaultWriteLatencyMs** - This is the default Latency trigger set for Reads and Writes given in milliseconds (ms), this is the threshold that is set to trigger a "Warning" state for the given database when those thresholds (one or both) are met. Think of this a minimum level of "Hey something may be wrong here".

@DefaultMinReads and @DefaultMinWrites - These values set the range of reads/writes and then averages out the latency between them. This helps scope down how narrow you want the test field to be for small chunks of I/O or if you have heavy I/O periods. This will vary from system to system but these can help adjust the sensitivity of your testing/alerting window to get more granular or more wide of a range of testing data.

@DefaultConsecutiveBreachesReq - This value is the number of times a concurrent threshold breach is allowed before an alert is sent, this primarily applies to Warning breaches and is one of the ways to help control alert noise due to common temporary Latency spikes.

@DefaultCooldownMinutes - This value set the amount of time (in seconds) before triggering another alert notification for the same database. This works in tandem with consecutive breaches to help mitigate creating a notification storm when thresholds are met.

@DefaultWarningWindowMinutes - This value measures the minutes between threshold breaches before triggering a warning notification, works in tandem with Consecutive Breaches. For example - 2 occurrences in 15 minutes.

@DefaultCriticalReadLatencyMs and @DefaultCriticalWriteLatencyMs - These values are the Critical (max allowed latency) before triggering a Critical alert. A critical alert is sent immediately regardless of above thresholds and indicates an event that should be looked into right away.

You will need to tweak these values as you begin to set this job up to function the way you want it too for your environment. Under the Default Policy are the defaults for every database, if you scroll down under the Thresholds/Policies section of the script below, you can call out specific databases to define specific thresholds for specific databases that you want to follow different thresholds for various reasons, for example such as TempDB which can be very noisy with defaults due to the nature of how it operates or SSRS/Reporting databases which are very heavy on writes but not so much on reads and so. You can add or remove any databases you like.

The other highlighted part of this job is the **@OperatorName** which will be the SQL Server Agent email operator you have setup on your server for alert notifications to be sent to.

The third and final job is the Disk Latency Snapshot Cleanup which will go out to both the Snapshot and Alert History tables and clean up old data to help prevent table bloat. The default retention periods are 14 days for Snapshots and 30 days for Alerts, anything older than that is purged. I have highlighted the values and you can adjust them as you like for your environment.

```
USE msdb;
GO

SET NOCOUNT ON;
GO

DECLARE @Category sysname = N'Database Maintenance';

IF NOT EXISTS
(
   SELECT 1
   FROM dbo.syscategories
   WHERE name = @Category
     AND category_class = 1
)
BEGIN
   EXEC dbo.sp_add_category
      @class = N'JOB',
      @type  = N'LOCAL',
      @name  = @Category;
END
GO

/*=========================================================
 Job 1: DBA - Disk Latency Snapshot Collector
=========================================================*/
IF EXISTS (SELECT 1 FROM dbo.sysjobs WHERE name = N'Disk Latency Snapshot Collector')
BEGIN
   EXEC dbo.sp_delete_job @job_name = N'Disk Latency Snapshot Collector';
END
GO
```

```
EXEC dbo.sp_add_job
   @job_name = N'Disk Latency Snapshot Collector',
   @enabled = 1,
   @description = N'Captures disk latency DMV snapshots into msdb.dbo.DiskLatencySnapshots.',
   @category_name = N'Database Maintenance',
   @owner_login_name = N'sa';
GO

EXEC dbo.sp_add_jobstep
   @job_name = N'Disk Latency Snapshot Collector',
   @step_name = N'Collect Snapshot',
   @subsystem = N'TSQL',
   @database_name = N'msdb',
   @command = N'
SET NOCOUNT ON;

INSERT dbo.DiskLatencySnapshots
(
   SnapshotTime,
   database_id,
   file_id,
   database_name,
   logical_name,
   physical_name,
   type_desc,
   num_of_reads,
   io_stall_read_ms,
   num_of_writes,
   io_stall_write_ms,
   size_on_disk_bytes,
   sample_ms
)
SELECT
   SYSDATETIME(),
   vfs.database_id,
   vfs.file_id,
   DB_NAME(vfs.database_id),
   mf.name,
   mf.physical_name,
   mf.type_desc,
   vfs.num_of_reads,
   vfs.io_stall_read_ms,
   vfs.num_of_writes,
   vfs.io_stall_write_ms,
   vfs.size_on_disk_bytes,
   vfs.sample_ms
FROM sys.dm_io_virtual_file_stats(NULL,NULL) vfs
JOIN sys.master_files mf
   ON mf.database_id = vfs.database_id
  AND mf.file_id = vfs.file_id
WHERE DB_NAME(vfs.database_id) IS NOT NULL;
',
   @on_success_action = 1,
   @on_fail_action = 2;
GO

EXEC dbo.sp_add_schedule
   @schedule_name = N'Disk Latency Snapshot Collector - Every 5 Minutes',
   @enabled = 1,
   @freq_type = 4,
   @freq_interval = 1,
   @freq_subday_type = 4,
   @freq_subday_interval = 5,
```

```
    @active_start_time = 000000;
GO

EXEC dbo.sp_attach_schedule
    @job_name = N'Disk Latency Snapshot Collector',
    @schedule_name = N'Disk Latency Snapshot Collector - Every 5 Minutes';
GO

EXEC dbo.sp_add_jobserver
    @job_name = N'Disk Latency Snapshot Collector';
GO

/*============================================================
  Job 2: DBA - Disk Latency Alert Monitor
============================================================*/
IF EXISTS (SELECT 1 FROM dbo.sysjobs WHERE name = N'Disk Latency Alert Monitor')
BEGIN
    EXEC dbo.sp_delete_job @job_name = N'Disk Latency Alert Monitor';
END
GO

EXEC dbo.sp_add_job
    @job_name = N'Disk Latency Alert Monitor',
    @enabled = 1,
    @description = N'Evaluates latest disk latency snapshot interval and sends alerts.',
    @category_name = N'Database Maintenance',
    @owner_login_name = N'sa';
GO

EXEC dbo.sp_add_jobstep
    @job_name = N'Disk Latency Alert Monitor',
    @step_name = N'Evaluate Snapshot Interval and Alert',
    @subsystem = N'TSQL',
    @database_name = N'msdb',
    @command = N'
SET NOCOUNT ON;

/*------------------------------------------------------------------------
  Default policy
------------------------------------------------------------------------*/
DECLARE @DefaultReadLatencyMs            int    = 140;
DECLARE @DefaultWriteLatencyMs           int    = 140;
DECLARE @DefaultMinReads                 bigint = 1000;
DECLARE @DefaultMinWrites                bigint = 8000;

DECLARE @DefaultConsecutiveBreachesReq     int    = 2;
DECLARE @DefaultCooldownMinutes            int    = 30;

DECLARE @DefaultWarningWindowMinutes       int    = 15;
DECLARE @DefaultCriticalReadLatencyMs      int    = 300;
DECLARE @DefaultCriticalWriteLatencyMs     int    = 300;

DECLARE @OperatorName sysname = N'SQL_Operator';
DECLARE @EmailSubject nvarchar(255) =
    N'SQL Disk Latency Alert on ' + CAST(SERVERPROPERTY('ServerName') as nvarchar(128));

IF OBJECT_ID('tempdb..#thresholds') IS NOT NULL DROP TABLE #thresholds;
IF OBJECT_ID('tempdb..#eval') IS NOT NULL DROP TABLE #eval;
IF OBJECT_ID('tempdb..#to_alert') IS NOT NULL DROP TABLE #to_alert;

/*------------------------------------------------------------------------
  Thresholds / policies
------------------------------------------------------------------------*/
CREATE TABLE #thresholds
```

```
(
    RuleOrder               int          NOT NULL,
    DbNamePattern           sysname      NOT NULL,
    MatchType               varchar(10)  NOT NULL,   -- EXACT | LIKE
    ReadLatencyMs           int          NOT NULL,
    WriteLatencyMs          int          NOT NULL,
    MinReads                bigint       NOT NULL,
    MinWrites               bigint       NOT NULL,
    ConsecutiveBreachesReq  int          NOT NULL,   -- warning occurrences required
    CooldownMinutes         int          NOT NULL,   -- applies to alerted events
    WarningWindowMinutes    int          NOT NULL,   -- rolling warning lookback
    CriticalReadLatencyMs   int          NOT NULL,   -- immediate alert
    CriticalWriteLatencyMs  int          NOT NULL    -- immediate alert
);

INSERT INTO #thresholds
(
    RuleOrder,
    DbNamePattern,
    MatchType,
    ReadLatencyMs,
    WriteLatencyMs,
    MinReads,
    MinWrites,
    ConsecutiveBreachesReq,
    CooldownMinutes,
    WarningWindowMinutes,
    CriticalReadLatencyMs,
    CriticalWriteLatencyMs
)
VALUES
    (1, N'%Reporting%', 'LIKE', 140, 140,  500,  500, 3, 30, 15, 350, 350),
    (2, N'tempdb',      'EXACT', 180, 300, 2000, 8000, 6, 30, 25, 800, 900);

/*-------------------------------------------------------------------------
  Get current vs previous snapshot
-------------------------------------------------------------------------*/
;WITH ranked AS
(
    SELECT
        s.SnapshotID,
        s.SnapshotTime,
        s.database_id,
        s.file_id,
        s.database_name,
        s.logical_name,
        s.physical_name,
        s.type_desc,
        s.num_of_reads,
        s.io_stall_read_ms,
        s.num_of_writes,
        s.io_stall_write_ms,
        ROW_NUMBER() OVER
        (
            PARTITION BY s.database_id, s.file_id
            ORDER BY s.SnapshotTime DESC, s.SnapshotID DESC
        ) AS rn
    FROM dbo.DiskLatencySnapshots s
),
curr_prev AS
(
    SELECT
        c.database_id,
        c.file_id,
```

```
        c.database_name,
        c.logical_name,
        c.physical_name,
        c.type_desc,
        c.SnapshotTime AS current_snapshot_time,
        p.SnapshotTime AS previous_snapshot_time,
        c.num_of_reads,
        p.num_of_reads AS prev_num_of_reads,
        c.io_stall_read_ms,
        p.io_stall_read_ms AS prev_io_stall_read_ms,
        c.num_of_writes,
        p.num_of_writes AS prev_num_of_writes,
        c.io_stall_write_ms,
        p.io_stall_write_ms AS prev_io_stall_write_ms
    FROM ranked c
    JOIN ranked p
      ON p.database_id = c.database_id
     AND p.file_id     = c.file_id
     AND p.rn          = 2
    WHERE c.rn = 1
),
calc AS
(
    SELECT
        database_id,
        file_id,
        database_name,
        logical_name,
        physical_name,
        type_desc,
        current_snapshot_time,
        previous_snapshot_time,
        num_of_reads - prev_num_of_reads AS reads_in_interval,
        num_of_writes - prev_num_of_writes AS writes_in_interval,
        CAST(
            (io_stall_read_ms - prev_io_stall_read_ms) * 1.0
            / NULLIF(num_of_reads - prev_num_of_reads, 0)
            AS decimal(18,2)
        ) AS avg_read_ms,
        CAST(
            (io_stall_write_ms - prev_io_stall_write_ms) * 1.0
            / NULLIF(num_of_writes - prev_num_of_writes, 0)
            AS decimal(18,2)
        ) AS avg_write_ms
    FROM curr_prev
    WHERE
        num_of_reads         >= prev_num_of_reads
        AND io_stall_read_ms  >= prev_io_stall_read_ms
        AND num_of_writes     >= prev_num_of_writes
        AND io_stall_write_ms >= prev_io_stall_write_ms
),
matched AS
(
    SELECT
        c.*,
        t.ReadLatencyMs,
        t.WriteLatencyMs,
        t.MinReads,
        t.MinWrites,
        t.ConsecutiveBreachesReq,
        t.CooldownMinutes,
        t.WarningWindowMinutes,
        t.CriticalReadLatencyMs,
        t.CriticalWriteLatencyMs,
```

```
      ROW_NUMBER() OVER
      (
        PARTITION BY c.database_id, c.file_id
        ORDER BY t.RuleOrder
      ) AS rule_rn
    FROM calc c
    LEFT JOIN #thresholds t
     ON
     (
       t.MatchType = 'EXACT'
       AND c.database_name = t.DbNamePattern
     )
     OR
     (
       t.MatchType = 'LIKE'
       AND c.database_name LIKE t.DbNamePattern
     )
),
final_eval AS
(
    SELECT
       database_id,
       file_id,
       database_name,
       logical_name,
       physical_name,
       type_desc,
       current_snapshot_time,
       previous_snapshot_time,
       reads_in_interval,
       writes_in_interval,
       avg_read_ms,
       avg_write_ms,
       COALESCE(ReadLatencyMs, @DefaultReadLatencyMs) AS read_threshold_ms,
       COALESCE(WriteLatencyMs, @DefaultWriteLatencyMs) AS write_threshold_ms,
       COALESCE(MinReads, @DefaultMinReads) AS min_reads_required,
       COALESCE(MinWrites, @DefaultMinWrites) AS min_writes_required,
       COALESCE(ConsecutiveBreachesReq, @DefaultConsecutiveBreachesReq) AS consecutive_breaches_required,
       COALESCE(CooldownMinutes, @DefaultCooldownMinutes) AS cooldown_minutes,
       COALESCE(WarningWindowMinutes, @DefaultWarningWindowMinutes) AS warning_window_minutes,
       COALESCE(CriticalReadLatencyMs, @DefaultCriticalReadLatencyMs) AS critical_read_threshold_ms,
       COALESCE(CriticalWriteLatencyMs, @DefaultCriticalWriteLatencyMs) AS critical_write_threshold_ms
    FROM matched
    WHERE rule_rn = 1 OR rule_rn IS NULL
)
SELECT
    e.*,
    CASE
       WHEN e.reads_in_interval  >= e.min_reads_required
        AND e.writes_in_interval >= e.min_writes_required
        AND e.avg_read_ms  >= e.read_threshold_ms
        AND e.avg_write_ms >= e.write_threshold_ms THEN N'BOTH'

       WHEN e.reads_in_interval  >= e.min_reads_required
        AND e.avg_read_ms  >= e.read_threshold_ms THEN N'READ'

       WHEN e.writes_in_interval >= e.min_writes_required
        AND e.avg_write_ms >= e.write_threshold_ms THEN N'WRITE'

       ELSE NULL
    END AS breach_type,
    CASE
       WHEN
          (
```

```
            e.reads_in_interval  >= e.min_reads_required
            AND e.avg_read_ms    >= e.critical_read_threshold_ms
          )
          OR
          (
            e.writes_in_interval >= e.min_writes_required
            AND e.avg_write_ms   >= e.critical_write_threshold_ms
          )
        THEN N'CRITICAL'

        WHEN
          (
            e.reads_in_interval  >= e.min_reads_required
            AND e.avg_read_ms    >= e.read_threshold_ms
          )
          OR
          (
            e.writes_in_interval >= e.min_writes_required
            AND e.avg_write_ms   >= e.write_threshold_ms
          )
        THEN N'WARNING'

        ELSE NULL
    END AS severity
INTO #eval
FROM final_eval e;

DELETE FROM #eval
WHERE breach_type IS NULL
   OR severity IS NULL;

/*--------------------------------------------------------------------------
  Decide which current events should alert
--------------------------------------------------------------------------*/
SELECT
    e.*,
    CAST(0 AS bit) AS should_alert
INTO #to_alert
FROM #eval e;

/*--------------------------------------------------------------------------
  Immediate alert for CRITICAL unless in cooldown from prior alerted event
--------------------------------------------------------------------------*/
UPDATE a
SET should_alert = 1
FROM #to_alert a
WHERE
    a.severity = N'CRITICAL'
    AND NOT EXISTS
    (
        SELECT 1
        FROM dbo.DiskLatencyAlertHistory h
        WHERE h.database_name = a.database_name
          AND h.logical_name  = a.logical_name
          AND h.type_desc     = a.type_desc
          AND h.breach_type   = a.breach_type
          AND h.WasAlerted    = 1
          AND h.AlertTime >= DATEADD(MINUTE, -a.cooldown_minutes, SYSDATETIME())
    );

/*--------------------------------------------------------------------------
  WARNING alerts only after repeated occurrences in rolling window
--------------------------------------------------------------------------*/
UPDATE a
```

```
SET should_alert = 1
FROM #to_alert a
WHERE
   a.severity = N'WARNING'
   AND
   (
      1 +
      (
         SELECT COUNT(*)
         FROM dbo.DiskLatencyAlertHistory h
         WHERE h.database_name = a.database_name
           AND h.logical_name  = a.logical_name
           AND h.type_desc     = a.type_desc
           AND h.breach_type   = a.breach_type
           AND h.Severity      = N'WARNING'
           AND h.AlertTime >= DATEADD(MINUTE, -a.warning_window_minutes, SYSDATETIME())
      )
   ) >= a.consecutive_breaches_required
   AND NOT EXISTS
   (
      SELECT 1
      FROM dbo.DiskLatencyAlertHistory h
      WHERE h.database_name = a.database_name
        AND h.logical_name  = a.logical_name
        AND h.type_desc     = a.type_desc
        AND h.breach_type   = a.breach_type
        AND h.WasAlerted    = 1
        AND h.AlertTime >= DATEADD(MINUTE, -a.cooldown_minutes, SYSDATETIME())
   );

/*--------------------------------------------------------------------------------
  Log ALL current events, whether alerted or suppressed
--------------------------------------------------------------------------------*/
INSERT dbo.DiskLatencyAlertHistory
(
   AlertTime,
   database_name,
   logical_name,
   type_desc,
   breach_type,
   avg_read_ms,
   avg_write_ms,
   reads_in_interval,
   writes_in_interval,
   WasAlerted,
   Severity,
   ConsecutiveBreachesRequired,
   CooldownMinutes,
   WarningWindowMinutes,
   CriticalReadThresholdMs,
   CriticalWriteThresholdMs
)
SELECT
   SYSDATETIME(),
   t.database_name,
   t.logical_name,
   t.type_desc,
   t.breach_type,
   t.avg_read_ms,
   t.avg_write_ms,
   t.reads_in_interval,
   t.writes_in_interval,
   t.should_alert,
   t.severity,
```

```
    t.consecutive_breaches_required,
    t.cooldown_minutes,
    t.warning_window_minutes,
    t.critical_read_threshold_ms,
    t.critical_write_threshold_ms
FROM #to_alert t;

/*--------------------------------------------------------------------------
  Build alert body from only rows selected for alert
--------------------------------------------------------------------------*/
IF EXISTS (SELECT 1 FROM #to_alert WHERE should_alert = 1)
BEGIN
    DECLARE @detail nvarchar(max);
    DECLARE @body   nvarchar(max);
    DECLARE @sep    nvarchar(200);

    SET @sep =
        CHAR(13) + CHAR(10) +
        N'----------------------------------------------------' +
        CHAR(13) + CHAR(10) + CHAR(13) + CHAR(10);

    SELECT @detail =
        STRING_AGG(
            CAST(
                CONCAT(
                    N'DB: ', COALESCE(e.database_name, N'(NULL)'),
                    N' | File: ', COALESCE(e.logical_name, N'(NULL)'),
                    N' | Type: ', COALESCE(e.type_desc, N'(NULL)'),
                    CHAR(13), CHAR(10),

                    N'Breach: ', COALESCE(e.breach_type, N'(NULL)'),
                    N' | Severity: ', COALESCE(e.severity, N'(NULL)'),
                    CHAR(13), CHAR(10),

                    N'Reads: ', COALESCE(CAST(e.reads_in_interval as nvarchar(30)), N'NULL'),
                    N' | Writes: ', COALESCE(CAST(e.writes_in_interval as nvarchar(30)), N'NULL'),
                    CHAR(13), CHAR(10),

                    N'Latency: Read ', COALESCE(CAST(e.avg_read_ms as nvarchar(30)), N'NULL'),
                    N' ms | Write ', COALESCE(CAST(e.avg_write_ms as nvarchar(30)), N'NULL'),
                    N' ms',
                    CASE
                        WHEN
                            e.read_threshold_ms            <> @DefaultReadLatencyMs
                            OR e.write_threshold_ms        <> @DefaultWriteLatencyMs
                            OR e.critical_read_threshold_ms  <> @DefaultCriticalReadLatencyMs
                            OR e.critical_write_threshold_ms <> @DefaultCriticalWriteLatencyMs
                            OR e.min_reads_required        <> @DefaultMinReads
                            OR e.min_writes_required       <> @DefaultMinWrites
                            OR e.consecutive_breaches_required <> @DefaultConsecutiveBreachesReq
                            OR e.warning_window_minutes    <> @DefaultWarningWindowMinutes
                            OR e.cooldown_minutes          <> @DefaultCooldownMinutes
                        THEN
                            CHAR(13) + CHAR(10) + CHAR(13) + CHAR(10) +
                            N'Overrides:' +
                            CHAR(13) + CHAR(10) +
                            N'  Warning  - Read > ' + COALESCE(CAST(e.read_threshold_ms as nvarchar(10)), N'NULL') +
                            N' ms | Write > ' + COALESCE(CAST(e.write_threshold_ms as nvarchar(10)), N'NULL') + N' ms' +
                            CHAR(13) + CHAR(10) +
                            N'  Critical - Read > ' + COALESCE(CAST(e.critical_read_threshold_ms as nvarchar(10)), N'NULL') +
                            N' ms | Write > ' + COALESCE(CAST(e.critical_write_threshold_ms as nvarchar(10)), N'NULL') + N' ms' +
                            CHAR(13) + CHAR(10) +
                            N'  Min Activity - Reads >= ' + COALESCE(CAST(e.min_reads_required as nvarchar(20)), N'NULL') +
                            N' | Writes >= ' + COALESCE(CAST(e.min_writes_required as nvarchar(20)), N'NULL') +
```

```
                CHAR(13) + CHAR(10) +
                N'  Alert Threshold - ' + COALESCE(CAST(e.consecutive_breaches_required as nvarchar(10)), N'NULL') +
                N' occurrences in ' + COALESCE(CAST(e.warning_window_minutes as nvarchar(10)), N'NULL') + N' min' +
                CHAR(13) + CHAR(10) +
                @sep +
                N'  Cooldown - ' + COALESCE(CAST(e.cooldown_minutes as nvarchar(10)), N'NULL') + N' min'
              ELSE N''
            END +
            CHAR(13) + CHAR(10) + CHAR(13) + CHAR(10) +

            N'Interval: ',
            COALESCE(CONVERT(nvarchar(19), e.previous_snapshot_time, 120), N'NULL'),
            N' -> ',
            COALESCE(CONVERT(nvarchar(19), e.current_snapshot_time, 120), N'NULL')
          ) AS nvarchar(max)
      ),
      @sep
    ) WITHIN GROUP
    (
      ORDER BY
        CASE e.severity
          WHEN N'CRITICAL' THEN 1
          WHEN N'WARNING'  THEN 2
          ELSE 3
        END,
        CASE e.breach_type
          WHEN N'BOTH'  THEN 1
          WHEN N'WRITE' THEN 2
          WHEN N'READ'  THEN 3
          ELSE 4
        END,
        ISNULL(e.avg_write_ms,0) DESC,
        ISNULL(e.avg_read_ms,0) DESC,
        e.database_name,
        e.logical_name
    )
FROM #to_alert e
WHERE e.should_alert = 1;

SET @body =
    N'Disk latency alert based on snapshot interval.' + CHAR(13)+CHAR(10) +
    CHAR(13)+CHAR(10) +
    N'Warnings require repeated occurrences.' + CHAR(13)+CHAR(10) +
    N'Critical events alert immediately.' + CHAR(13)+CHAR(10) +
    CHAR(13)+CHAR(10) +
    N'Default thresholds:' + CHAR(13)+CHAR(10) +
    N'  Warning  - Read > ' + CAST(@DefaultReadLatencyMs as nvarchar(10)) +
    N' ms | Write > ' + CAST(@DefaultWriteLatencyMs as nvarchar(10)) + N' ms' + CHAR(13)+CHAR(10) +
    N'  Critical - Read > ' + CAST(@DefaultCriticalReadLatencyMs as nvarchar(10)) +
    N' ms | Write > ' + CAST(@DefaultCriticalWriteLatencyMs as nvarchar(10)) + N' ms' + CHAR(13)+CHAR(10) +
    N'  Min Activity - Reads >= ' + CAST(@DefaultMinReads as nvarchar(20)) +
    N' | Writes >= ' + CAST(@DefaultMinWrites as nvarchar(20)) + CHAR(13)+CHAR(10) +
    CHAR(13)+CHAR(10) +
    N'Default alert threshold: ' + CAST(@DefaultConsecutiveBreachesReq as nvarchar(10)) +
    N' occurrences in ' + CAST(@DefaultWarningWindowMinutes as nvarchar(10)) + N' min' + CHAR(13)+CHAR(10) +
    N'Default cooldown: ' + CAST(@DefaultCooldownMinutes as nvarchar(10)) + N' min' + CHAR(13)+CHAR(10) +
    CHAR(13)+CHAR(10) +
    @sep +
    COALESCE(@detail, N'(No detail rows generated)');

EXEC dbo.sp_notify_operator
    @name    = @OperatorName,
    @subject = @EmailSubject,
    @body    = @body;
```

```
END;
',
    @on_success_action = 1,
    @on_fail_action = 2;
GO

EXEC dbo.sp_add_schedule
    @schedule_name = N'Disk Latency Alert Monitor - Every 5 Minutes',
    @enabled = 1,
    @freq_type = 4,
    @freq_interval = 1,
    @freq_subday_type = 4,
    @freq_subday_interval = 5,
    @active_start_time = 0000 00;
GO

EXEC dbo.sp_attach_schedule
    @job_name = N'Disk Latency Alert Monitor',
    @schedule_name = N'Disk Latency Alert Monitor - Every 5 Minutes';
GO

EXEC dbo.sp_add_jobserver
    @job_name = N'Disk Latency Alert Monitor';
GO

/*============================================================
  Job 3: DBA - Disk Latency Snapshot Cleanup
============================================================*/
IF EXISTS (SELECT 1 FROM dbo.sysjobs WHERE name = N'Disk Latency Snapshot Cleanup')
BEGIN
    EXEC dbo.sp_delete_job @job_name = N'Disk Latency Snapshot Cleanup';
END
GO

EXEC dbo.sp_add_job
    @job_name = N'Disk Latency Snapshot Cleanup',
    @enabled = 1,
    @description = N'Removes old disk latency snapshots and alert history.',
    @category_name = N'Database Maintenance',
    @owner_login_name = N'sa';
GO

EXEC dbo.sp_add_jobstep
    @job_name = N'Disk Latency Snapshot Cleanup',
    @step_name = N'Cleanup Old Disk Latency Data',
    @subsystem = N'TSQL',
    @database_name = N'msdb',
    @command = N'
SET NOCOUNT ON;

DECLARE @SnapshotRetentionDays int = 14;
DECLARE @AlertHistoryRetentionDays int = 30;

DELETE FROM dbo.DiskLatencySnapshots
WHERE SnapshotTime < DATEADD(DAY, -@SnapshotRetentionDays, SYSDATETIME());

DELETE FROM dbo.DiskLatencyAlertHistory
WHERE AlertTime < DATEADD(DAY, -@AlertHistoryRetentionDays, SYSDATETIME());
',
    @on_success_action = 1,
    @on_fail_action = 2;
GO

EXEC dbo.sp_add_schedule
```

```
    @schedule_name = N'Disk Latency Snapshot Cleanup - Daily',
    @enabled = 1,
    @freq_type = 4,
    @freq_interval = 1,
    @active_start_time = 001500;
GO

EXEC dbo.sp_attach_schedule
    @job_name = N'Disk Latency Snapshot Cleanup',
    @schedule_name = N'Disk Latency Snapshot Cleanup - Daily';
GO

EXEC dbo.sp_add_jobserver
    @job_name = N'Disk Latency Snapshot Cleanup';
GO
```

Once setup your alert emails (if you receive any), should look similar to this…

```
Subject:
SQL Disk Latency Alert on SERVER_NAME

Body:
Disk latency alert based on snapshot interval.

Warnings require repeated occurrences.
Critical events alert immediately.

Default thresholds:
  Warning  - Read > 140 ms | Write > 140 ms
  Critical - Read > 300 ms | Write > 300 ms
  Min Activity - Reads >= 2000 | Writes >= 2000

Default alert threshold: 2 occurrences in 15 min
Default cooldown: 30 min

----------------------------------------------------

DB: SampleDB | File: SampleDB_Data | Type: ROWS
Breach: WRITE | Severity: CRITICAL
Reads: 3200 | Writes: 18450
Latency: Read 95.00 ms | Write 412.80 ms

Interval: 2026-04-07 09:10:00 -> 2026-04-07 09:15:00

----------------------------------------------------

DB: ReportingDB | File: ReportingDB | Type: ROWS
Breach: BOTH | Severity: WARNING
Reads: 7800 | Writes: 2600
Latency: Read 182.40 ms | Write 158.20 ms

Overrides:
  Warning  - Read > 140 ms | Write > 140 ms
  Critical - Read > 350 ms | Write > 350 ms
  Min Activity - Reads >= 500 | Writes >= 500
  Alert Threshold - 3 occurrences in 15 min
  Cooldown - 30 min

Interval: 2026-04-07 09:10:00 -> 2026-04-07 09:15:00
```

Step 3 - Table Checks (Optional)

Below are a couple commands to check the 2 tables if desired. The Alert History table especially could be useful if you are investigating re-

occurring/ongoing latency issues with particular databases or as a whole. The Snapshot table is more as a reference point or to confirm operationally the collector is working.

Snapshot Table Query

```
SELECT TOP (1000) [SnapshotID]
    ,[SnapshotTime]
    ,[database_id]
    ,[file_id]
    ,[database_name]
    ,[logical_name]
    ,[physical_name]
    ,[type_desc]
    ,[num_of_reads]
    ,[io_stall_read_ms]
    ,[num_of_writes]
    ,[io_stall_write_ms]
    ,[size_on_disk_bytes]
    ,[sample_ms]
  FROM [msdb].[dbo].[DiskLatencySnapshots]
```

Alert History Table Query

```
SELECT TOP (1000) [AlertHistoryID]
    ,[AlertTime]
    ,[database_name]
    ,[logical_name]
    ,[type_desc]
    ,[breach_type]
    ,[avg_read_ms]
    ,[avg_write_ms]
    ,[reads_in_interval]
    ,[writes_in_interval]
    ,[WasAlerted]
    ,[Severity]
    ,[ConsecutiveBreachesRequired]
    ,[CooldownMinutes]
    ,[WarningWindowMinutes]
    ,[CriticalReadThresholdMs]
    ,[CriticalWriteThresholdMs]
  FROM [msdb].[dbo].[DiskLatencyAlertHistory]
```

Clear Tables (if Needed)

This is more for an "as needed" scenario but if for some reason (testing most likely) you want to clear the data from these tables and start fresh, run these commands. Keep in mind, this will completed clean all entries from these two tables so it will remove all historical and current data and start collection from scratch.

```
TRUNCATE TABLE msdb.dbo.DiskLatencyAlertHistory;
TRUNCATE TABLE msdb.dbo.DiskLatencySnapshots;
```